

Simulazione di un sistema a N-corpi

Lorenzo Facchini

2 Luglio 2026

1 Scelte Progettuali e Implementative

Il programma consta di una classe principale chiamata **App**. Essa opportunamente inizializzata da file di configurazione, gestisce tramite due classi distinte (**Simulation** e **Renderer**) la simulazione fisica e la rappresentazione grafica della simulazione. Per il calcolo delle posizioni, velocità e accelerazioni dei corpi è stata implementata una struct (**Vec2**) nella quale sono state definite le operazioni utili tra i vettori in due dimensioni e si è sfruttato l'algoritmo Velocity Verlet per il calcolo delle orbite. Nella classe **Body** sono conservate la posizione, velocità, accelerazione, massa e id per ogni corpo coinvolto nella simulazione. La rappresentazione dell'evoluzione del moto dei corpi nel tempo è affidata alla classe **Renderer**, la quale, sfruttando le classi e le funzioni della libreria esterna SFML, disegna su una finestra dei cerchi per rappresentare i corpi. Questi assumono una posizione diversa ogni frame, ottenuta dal calcolo effettuato con i metodi di **Simulation**. La simulazione può terminare quando esegue un numero prestabilito dall'utente di steps oppure chiudendo la finestra grafica. Tutte le classi, struct e funzioni sono dichiarate all'interno di un unico namespace **nc**.

1.1 App.hpp

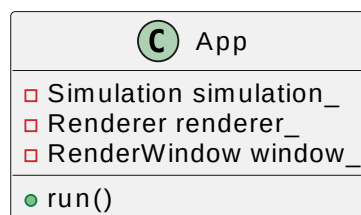


Figura 1: Diagramma di App.

Nel file **App.hpp** è dichiarata la classe **App**, che gestisce la finestra di visualizzazione della simulazione tramite **sf::RenderWindow** di SFML, inizializzata con i parametri del **RendererConfig**. La classe contiene inoltre un oggetto **Simulation**, che controlla l'evoluzione fisica, e un oggetto **Renderer**, responsabile del rendering degli oggetti simulati. Entrambi sono costruiti a partire dalle rispettive configurazioni.

Il metodo **run()** implementa il **game loop** principale dell'applicazione. All'interno del ciclo viene utilizzato un oggetto **sf::Clock** per misurare il tempo reale trascorso tra un frame e

il successivo. Questo tempo viene moltiplicato per un fattore `time_scale_`, che consente di accelerare o rallentare la simulazione rispetto al tempo reale. La funzione si occupa anche degli eventi della finestra, inclusa la chiusura o il termine della simulazione.

La simulazione avanza a passi discreti tramite `simulation_.step()` quando il tempo accumulato supera `simulation_.dt()`. Ad ogni iterazione la scena viene aggiornata con il `Renderer` e mostrata tramite `window_.display()`.

1.2 Simulation.hpp

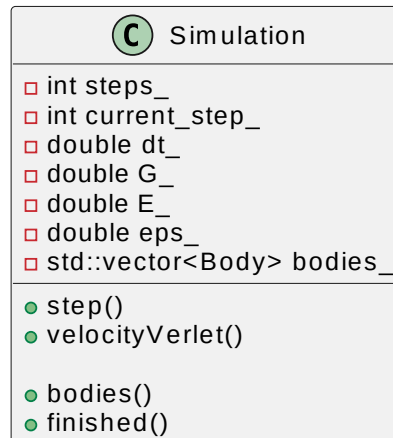


Figura 2: Diagramma di Simulation.

Nel file `simulation.hpp` sono contenuti la classe `Simulation` e le funzioni libere `computeAccelerations()`, `kineticEnergy()`, `potentialEnergy()` e `totalEnergy()`. La classe si occupa di gestire la parte fisica del programma. Essa possiede dati utili alla simulazione, conservati nella parte `private`, quali: il numero di steps da calcolare e oggetti di `Body` che contengono i dati relativi ai corpi della simulazione. Questi ultimi sono allocati in un vettore della classe `std::vector` standard library. La classe consta di quattro metodi, tra cui `velocityVerlet()` che, seguendo l'omonimo algoritmo, calcola le posizioni, velocità e accelerazioni dei corpi, e `finished()` che quando invocato controlla se lo step corrente della simulazione ha superato quello massimo prefissato, restituendo in pratica se la simulazione deve terminare. Le funzioni non membro sono invece: `computeAccelerations()`, la quale serve per calcolare con un ciclo separato l'accelerazione di un corpo rispetto agli altri; `kineticEnergy()`, `potentialEnergy()` e `totalEnergy()` che servono a calcolare l'energia meccanica totale del sistema, ottenuta dalla somma dell'energia cinetica e potenziale.

1.3 Body.hpp

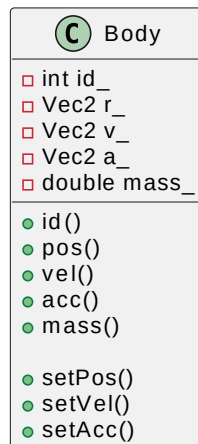


Figura 3: Diagramma di Body.

Il file in questione contiene la classe **Body**, che si occupa di conservare i dati relativi a posizione, velocità accelerazione, massa e un numero identificativo `id`. Non possiede metodi importanti se non quelli per modificare i dati cinematici del corpo.

1.4 Vec2.hpp

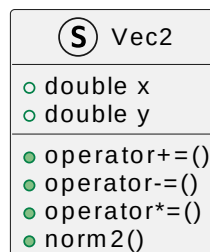


Figura 4: Diagramma di Vec2.

Nel file `vec2.hpp` è dichiarata la struct **Vec2**, la quale si occupa di definire la coppia ordinata (x, y) e le regole di calcolo tra elementi dello stesso tipo. Sono inoltre presenti le funzioni libere (overload) per gestire: `operator+`, `operator-`, `operator*`

1.5 Renderer.hpp

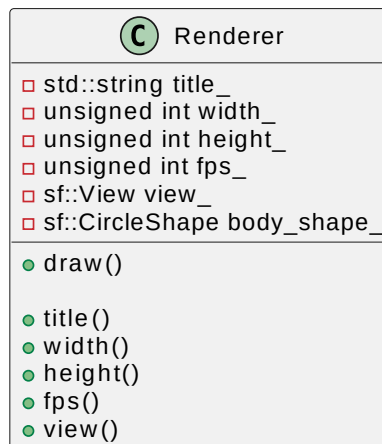


Figura 5: Diagramma di `Renderer`.

Nel file `renderer.hpp` è dichiarata l'omonima classe **Renderer**, che si occupa di gestire, insieme ad **App**, la parte grafica del programma. Essa contiene i dati relativi alla camera (derivata da `sf::View`), alla forma dei corpi (oggetto di `sf::CircleShape`), alle dimensioni della finestra e altri parametri grafici. Il metodo più importante è `draw()` che disegna le forme dei corpi seguendo le coordinate proprie dell'oggetto.

1.6 SimulationConfig.hpp

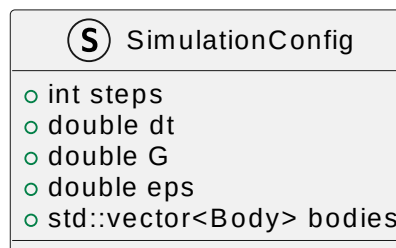


Figura 6: Diagramma di `SimulationConfig`.

Nel file `simulationConfig.hpp` è dichiarata la struct **SimulationConfig**, che serve per inizializzare la classe Simulazione ottenendo i dati da un file di configurazione. Contiene i parametri necessari all'inizializzazione della simulazione, letti da file di configurazione.

1.7 RendererConfig.hpp

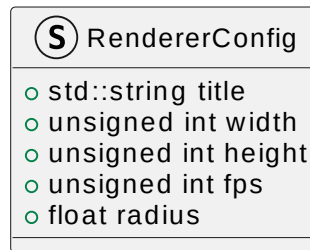


Figura 7: Diagramma di RendererConfig.

Nel file `rendererConfig.hpp` è dichiarata la struct `RendererConfig` serve per inizializzare la classe `Renderer` ottenendo i dati da un file di configurazione. Contiene le variabili relative alle dimensioni della finestra, il titolo e altri parametri della simulazione.

1.8 ConfigLoader.hpp

Nel file `configLoader.hpp` sono dichiarate le funzioni responsabili per il caricamento dei dati della simulazione da file configurazionali ad una struct di configurazione. Le funzioni codificate sono due: `loadSimulation()`, responsabile per l'inizializzazione di `SimulationConfig` e `loadRenderer()`, responsabile per l'inizializzazione di `RendererConfig`. Tali funzioni accettano un file di configurazione predisposto, il quale viene prima pulito di commenti e righe vuote, poi sono estratti i dati contenuti. Le funzioni svolgono anche il controllo che il valore in input sia accettabile.

1.9 File di configurazione

I file di configurazione hanno tutti una stessa struttura simile. Ogni riga possiede una parola iniziale, chiamata *key*, che dichiara quale grandezza rappresenta la stringa che la segue. Ad eccezione delle *keys* "body" nel file di configurazione di `SimulationConfig` e "title" nel file di configurazione di `RendererConfig` nessuna riga può contenere più di due stringhe di caratteri separati da spazi, altrimenti sarà lanciato un errore. I file possono però essere commentati con il carattere "#" e tutto quello che segue sarà ignorato.

Le key accettate per il file di configurazione di `SimulationConfig` sono:

- "N" numero di corpi coinvolti nella simulazione;
- "dt" delta time della simulazione, aumentandolo si aumenta la precisione dei calcoli, utile quando l'energia aumenta troppo;
- "steps" numero di step da calcolare nella simulazione;
- "G" costante di gravitazione universale da usare nei calcoli;
- "eps" parametro di softening, più è piccolo e meno incide nei conti;
- "body" dichiara che i dati che seguono sono di un oggetto `Body`. Si aspetta quindi cinque numeri che rappresentano massa, posizioni iniziali x e y, velocità iniziali x e y.

Le key accettate per il file di configurazione di `RendererConfig` sono:

- "title" tutti i caratteri che seguono questa key saranno inseriti nel titolo della finestra grafica;
- "width" larghezza della finestra;
- "height" altezza della finestra;
- "fps" limite degli FPS della finestra;
- "radius" raggio dei cerchi che rappresentano i corpi nella finestra.

2 Dipendenze e Librerie Esterne

Il progetto è compilato mediante CMake e utilizza la libreria SFML per la visualizzazione grafica, quindi è necessario averla installata sul proprio computer. Si può eseguire questo comando:

```
1 sudo apt install libsFML-dev
```

Per i test automatici viene impiegata la libreria doctest.

3 Istruzioni per l'Esecuzione

Per compilare il progetto:

```
1 cmake -S . -B build -G"Ninja Multi-Config"
2 cmake --build build --config Debug
3 cmake --build build --config Debug --target test
4 cmake --build build --config Release
5 cmake --build build --config Release --target test
```

Per eseguire il programma si può scegliere uno dei file di configurazione predisposti (situati nella cartella `config/`, oppure crearne uno con valori personalizzati. In ogni caso il nome del file di configurazione, che deve contenere tutti i dati necessari descritti approfonditamente nella sezione 4, si deve passare come argomento quando si digita il comando dell'eseguibile. Il secondo argomento, opzionale, è `time_scale`, che rappresenta il parametro per velocizzare o rallentare la simulazione. Il suo intervallo di valori consentito è $0 < \text{time_scale} < 10$. L'inserimento di valori al di fuori di questo intervallo o di caratteri non numerici genererà un errore. Qualora questo parametro non venisse specificato, gli verrà assegnato il valore predefinito di 1.

Nelle seguenti istruzioni, a titolo di esempio, la simulazione assume i dati tipici della Figure-8 e altri.

```
1 build/Debug/nbodysim config/simulation-8-shape.config
2 build/Release/nbodysim config/simulation-8-shape.config 1.5
3 build/Debug/nbodysim config/lagrange_L4.config .1
4 build/Debug/nbodysim config/random.config
5 build/Debug/nbodysim config/solar-like.config .2
```

L'esecuzione dei test, invece, non richiede alcun file di configurazione e si possono lanciare con i comandi:

```
1 build/Debug/nbodysim.t
2 build/Release/nbodysim.t
```

4 Input e Output

Il programma esegue la simulazione partendo da un file di configurazione che contiene i dati necessari per avviarla. Sono due i file di configurazione necessari: `simulation-<name>.config` e `renderer.config`.

Il primo file contiene tutti i dati relativi alla simulazione fisica. Di seguito un esempio con spiegazione riga per riga.

```
1 # Parametri simulazione Figure-8
2 N 3                      # numero di corpi
3 dt 0.001                 # delta time
4 steps 10000              # numero di step fisici della simulazione
5 G 1.0                    # costante di gravitazione universale
6 eps 1e-12                # parametro di softening
7
8 # Corpi
9 # body massa pos_x pos_y vel_x vel_y
10 body 1.0 -0.97000436 0.24308753 0.4662036850 0.4323657300
11 body 1.0 0.97000436 -0.24308753 0.4662036850 0.4323657300
12 body 1.0 0.0 0.0 -0.93240737 -0.86473146
```

Il codice identificativo proprio di ogni oggetto Body è assegnato automaticamente dal programma e non si deve scrivere nel file di configurazione.

Il secondo file contiene i dati relativi alla rappresentazione grafica della simulazione. Esso non è passabile come argomento lanciando l'eseguibile con il comando di shell. I dati si possono però modificare andando a cambiare i valori nel file di configurazione situato nella cartella `config/`. Di seguito è riportato il file usato nel progetto commentato riga per riga.

```
1 # Parametri finestra SFML
2 title N-Body Simulator    # titolo della finestra
3 width 800                 # larghezza della finestra
4 height 600                # altezza della finestra
5 fps 60                    # FPS massimi
6 radius 0.1f               # raggio dei cerchi che rappresentano i corpi
```

Nei file di configurazione è possibile commentare ovunque con `"#"`. Il testo che segue `"#"` non è considerato dal programma.

Durante il caricamento vengono effettuati controlli di validità sul file di configurazione; in caso di errore viene generata un'eccezione che riporta il nome del file e, quando possibile, il numero della riga.

L'output prodotto sarà l'apertura di una finestra con la rappresentazione grafica della simulazione, altrimenti un messaggio d'errore nel caso qualcosa non fosse andato correttamente

come: dati incorretti o non completi, non conservazione dell'energia, ecc.

5 Interpretazione dei Risultati

L'esecuzione del programma provoca l'apertura di una finestra che mostra l'evoluzione del sistema N-Body nel tempo calcolata con l'algoritmo velocity verlet. La simulazione procede fino al raggiungimento di numero massimo di steps fisici, che si possono impostare nel file di configurazione per la simulazione.

Durante la simulazione viene monitorata la conservazione dell'energia meccanica totale. Se lo scarto relativo tra l'energia iniziale e quella corrente supera il 5%, il programma interrompe la simulazione generando un'eccezione, poiché tale condizione indica che gli errori numerici sono diventati significativi.

Vale la pena menzionare che assegnando a `dt` troppo grande la simulazione porterà un errore di calcolo grande, causando la non conservazione dell'energia.

6 Strategia di Test

Per verificare la correttezza del programma è stata adottata una strategia di unit testing mediante la libreria `doctest`. Ogni componente è stato testato in modo indipendente, verificandone sia il corretto funzionamento in condizioni normali sia la gestione di input non validi e delle relative eccezioni.

6.1 Struct Vec2

Sono stati verificati gli operatori aritmetici (+, -, * e le rispettive versioni di assegnamento), l'operatore di uguaglianza e il calcolo della norma al quadrato. I test comprendono vettori nulli, valori positivi e negativi e confronti tra risultati ottenuti con differenti operatori.

6.2 Classe Body

Sono stati verificati i metodi getter e setter, controllando che posizione, velocità, accelerazione e massa vengano inizializzati e modificati correttamente.

6.3 Funzioni libere in `simulation.hpp` e funzioni membro di `Simulation`

Sono state testate le funzioni che implementano i principali calcoli fisici: `kineticEnergy()`, `potentialEnergy()`, `totalEnergy()` e `computeAccelerations()`. `Simulation::velocityVerlet()` Per ciascuna funzione sono stati costruiti semplici sistemi di uno, due e tre corpi, confrontando i risultati ottenuti con valori calcolati manualmente. I confronti sui numeri in virgola mobile sono stati effettuati mediante `doctest::Approx`, così da tenere conto degli errori di arrotondamento.

6.4 Caricamento dei file di configurazione

Sono state testate le funzioni `loadSimulation()` e `loadRender()` mediante file di configurazione sia validi sia volutamente errati. In particolare sono stati verificati: il corretto caricamento di tutti i parametri; la gestione di file inesistenti; la presenza di chiavi non riconosciute; la mancanza di parametri obbligatori; valori con formato non valido; la corretta generazione delle eccezioni con messaggi di errore contenenti il nome del file e, quando disponibile, il numero della riga. Viene anche controllata la coerenza tra il numero di corpi dichiarati e quelli poi definiti.

6.5 Funzioni di conversione

Sono state testate le funzioni: `stringToInt()`, `stringToDouble()` e `stringToSize_t()`, verificando sia la corretta conversione di valori validi sia la gestione di stringhe non numeriche, stringhe vuote e casi di overflow.

7 Utilizzo di Sistemi di Intelligenza Artificiale Generativa

I sistemi di Intelligenza Artificiale Generativa sono stati usati principalmente per: analizzare i messaggi d'errore del compilatore per comprendere gli errori nel codice, risolvere problemi di architettura software, debugging per errori logici.

8 Informazioni Aggiuntive

È possibile variare il `dt` per aumentare la precisione nel calcolo il sistema lo richiede. Inoltre è consigliabile impostare un corretto fattore di scala, ad esempio 0.2 o 0.1, sia se l'output visivo risulta troppo veloce sia se il sistema contiene molti corpi gravando sul costo di calcolo.